Practical 1: Exploratory Data Analysis and Data Visualisation

Executed notebook report

Code Cell 1

```
# Practical 1 : program to perform exploratory data analysis and data visualisation
#import required libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
(No output)
```

Code Cell 2

```
# Load Titanic dataset
df = pd.read_csv(r"/Users/pratikvilasgadhe/Desktop/Programming/AI_ML/titanic/train.csv")
print("Dataset Loads Sucessfully!")
```

Output

```
Dataset Loads Sucessfully!
```

Code Cell 3

```
print("EXPLORATORY DATA ANALYSIS (EDA)")
# Dataset overview
print("\n--- Dataset Info ---")
df.info()
```

Output

```
EXPLORATORY DATA ANALYSIS (EDA)

--- Dataset Info ---
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

Code Cell 4

```
print("\n--- Summary Statistics (Numerical Features) ---")
print(df.describe())
```

Output

```
--- Summary Statistics (Numerical Features) ---
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
```

```
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
```

Code Cell 5

```
print("\n--- Missing Values Check ---")
missing_values = df.isnull().sum()
print(missing_values[missing_values > 0])
```

Output

```
--- Missing Values Check ---
Age         177
Cabin       687
Embarked      2
dtype: int64
```

Code Cell 6

```
print("\n--- Duplicate Values Check ---")
duplicated_values = df.duplicated().sum()
print(duplicated_values[duplicated_values > 0])
```

Output

```
--- Duplicate Values Check ---
[]
```

Code Cell 7

```
print("\n--- Summary Statistics (Object Features) ---")
print(df.describe(include=[object]))
```

Output

```
--- Summary Statistics (Object Features) ---
                           Name   Sex  Ticket Cabin Embarked
count                       891   891     891   204      889
unique                      891     2     681   147        3
top     Braund, Mr. Owen Harris  male  347082    G6        S
freq                          1   577       7     4      644

/var/folders/gq/pn_jxryd2kv7467xp_wyc6v80000gn/T/ipykernel_25121/794624.py:2: Pandas4Warning: For backward compatibility, 'str' d
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write
  print(df.describe(include=[object]))
```

Code Cell 8

```
print("\n--- Survival Rate by Passenger Class ---")
if "Pclass" in df.columns and "Survived" in df.columns:
    print(df.groupby("Pclass")["Survived"].mean())
```

Output

```
--- Survival Rate by Passenger Class ---
Pclass
1    0.629630
2    0.472826
3    0.242363
Name: Survived, dtype: float64
```

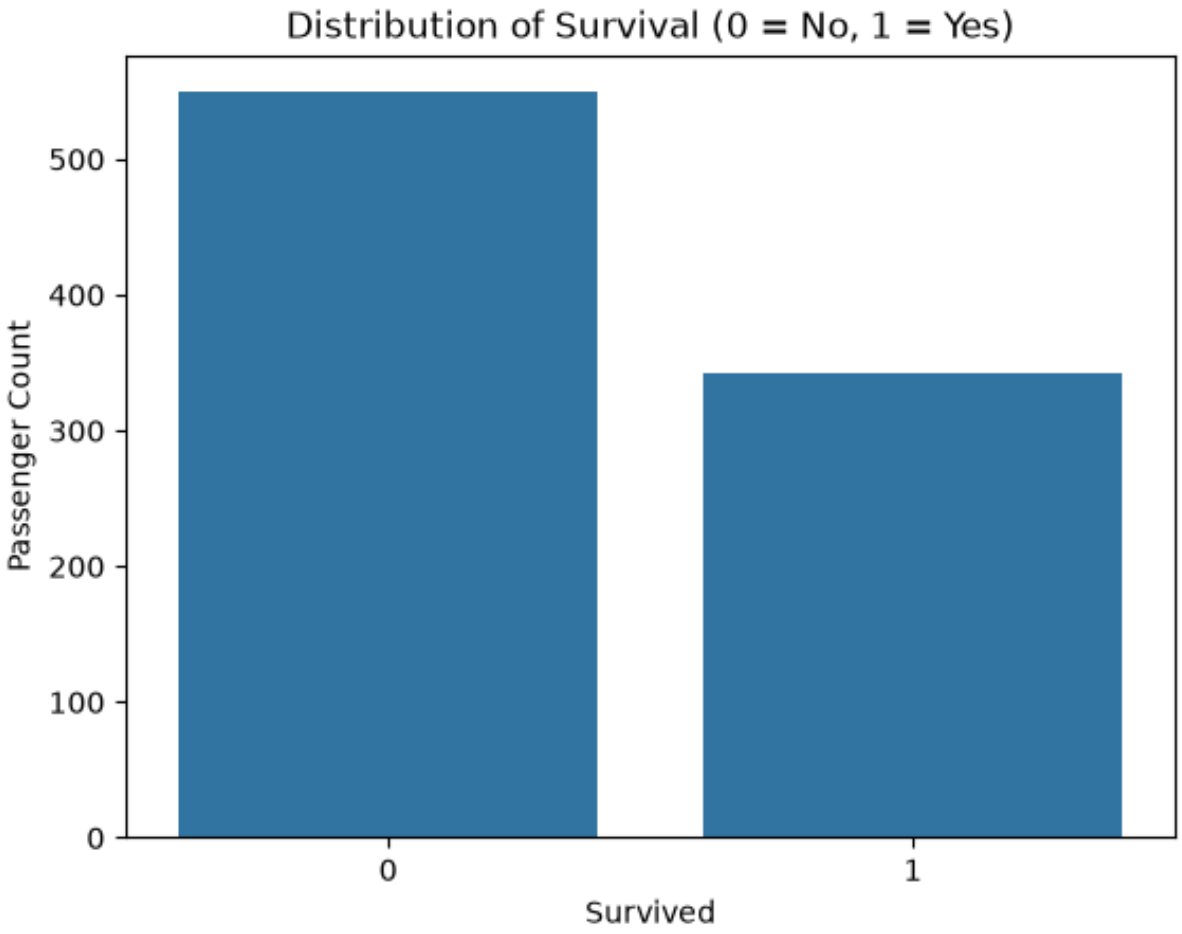
Code Cell 9

```
print("DATA VISUALIZATION")
# 1. Survival Count Plot
plt.figure()
sns.countplot(data=df, x=df['Survived'])
plt.title("Distribution of Survival (0 = No, 1 = Yes)")
plt.xlabel("Survived")
```

```
plt.ylabel("Passenger Count")
plt.savefig("titanic_survival_count.png")
plt.show()
plt.close()
print("Saved: titanic_survival_count.png")
```

Output

```
DATA VISUALIZATION
```



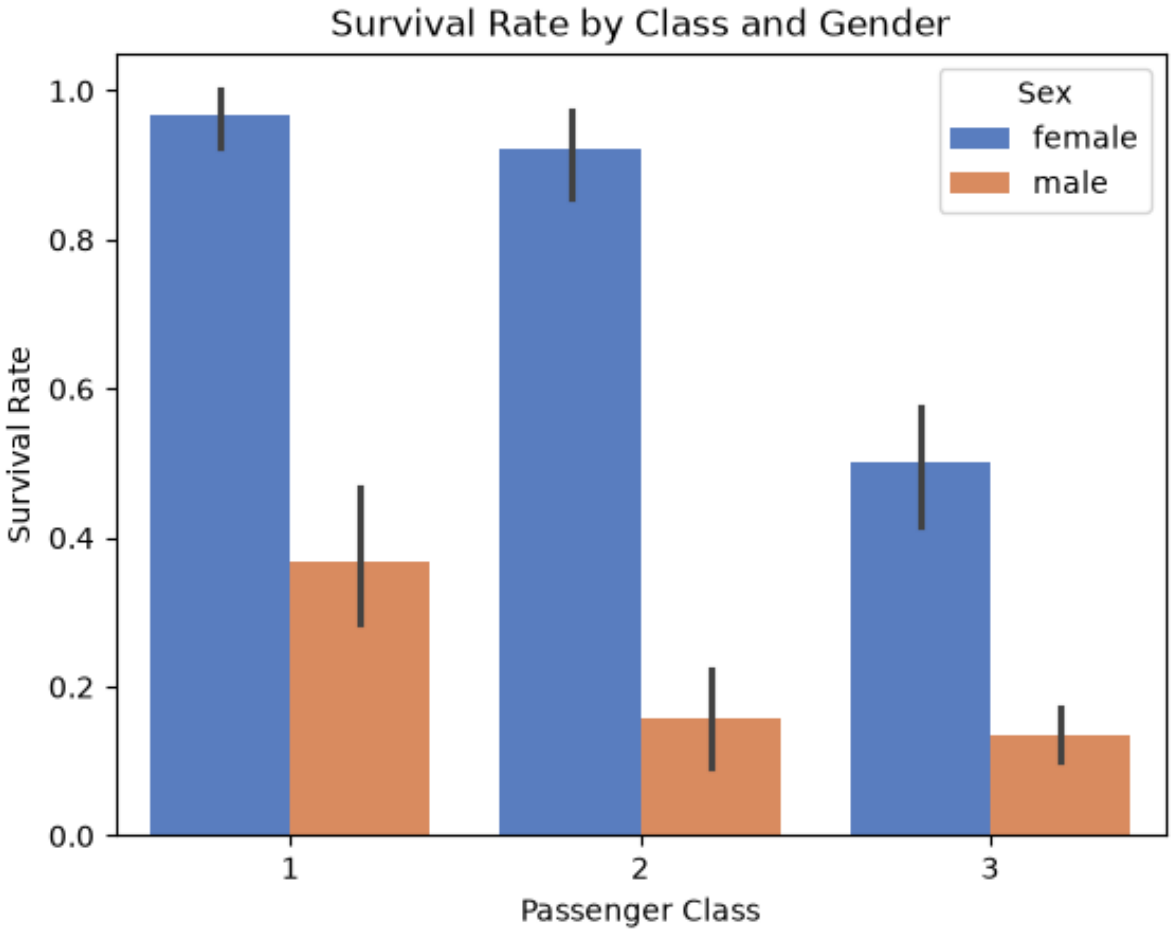

```
<Figure size 640x480 with 1 Axes>
```

```
Saved: titanic_survival_count.png
```

Code Cell 10

```
# 2. Survival Rate by Sex and Class
plt.figure()
sns.barplot(data=df, x=df['Pclass'], y=df['Survived'], hue=df['Sex'], palette="muted")
plt.title("Survival Rate by Class and Gender")
plt.xlabel("Passenger Class")
plt.ylabel("Survival Rate")
plt.savefig("titanic_survival_by_class_sex.png")
plt.show()
plt.close()
print("Saved: titanic_survival_by_class_sex.png")
```

Output



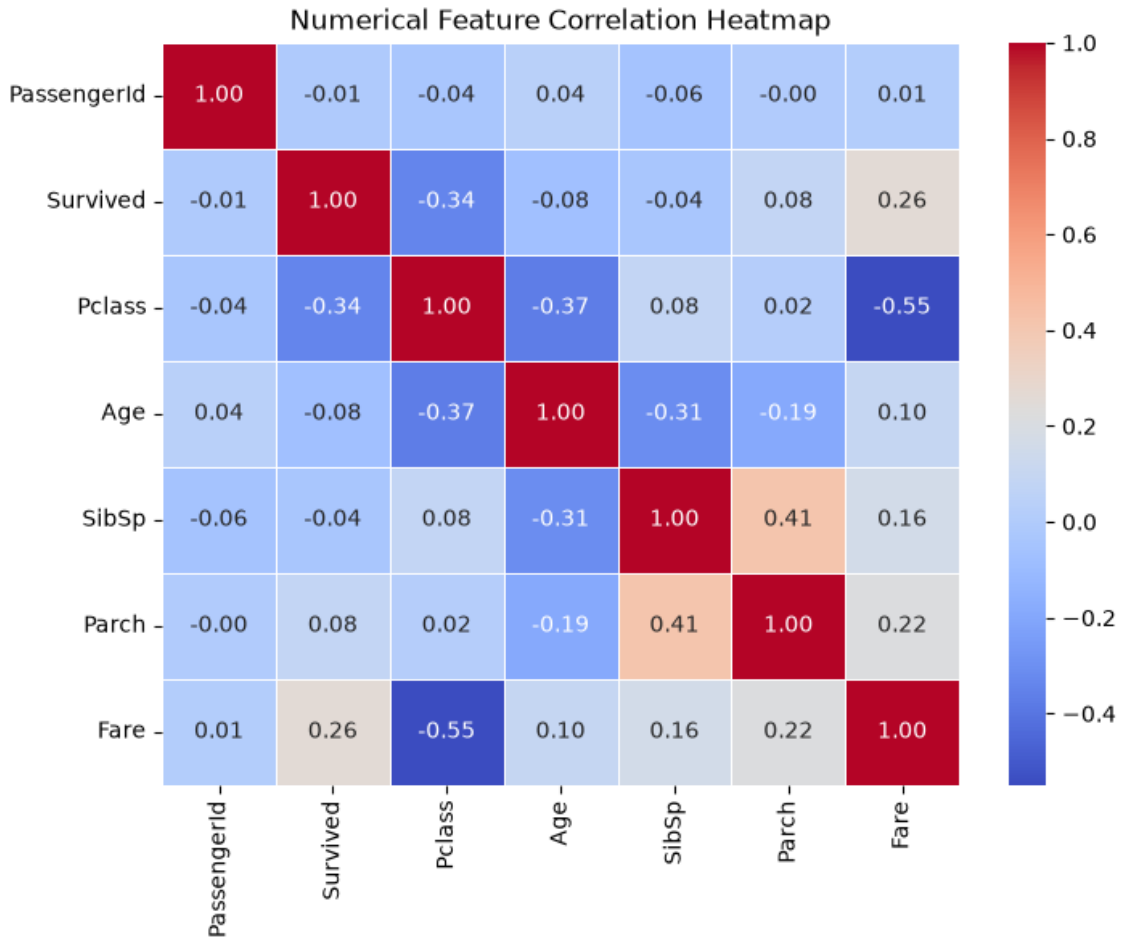

```
<Figure size 640x480 with 1 Axes>
```

```
Saved: titanic_survival_by_class_sex.png
```

Code Cell 11

```
# 3. Correlation Heatmap for Numerical Features
plt.figure(figsize=(8, 6))
numeric_df = df.select_dtypes(include=[np.number])
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Numerical Feature Correlation Heatmap")
plt.savefig("titanic_correlation_heatmap.png")
plt.show()
plt.close()
print("Saved: titanic_correlation_heatmap.png")
```

Output



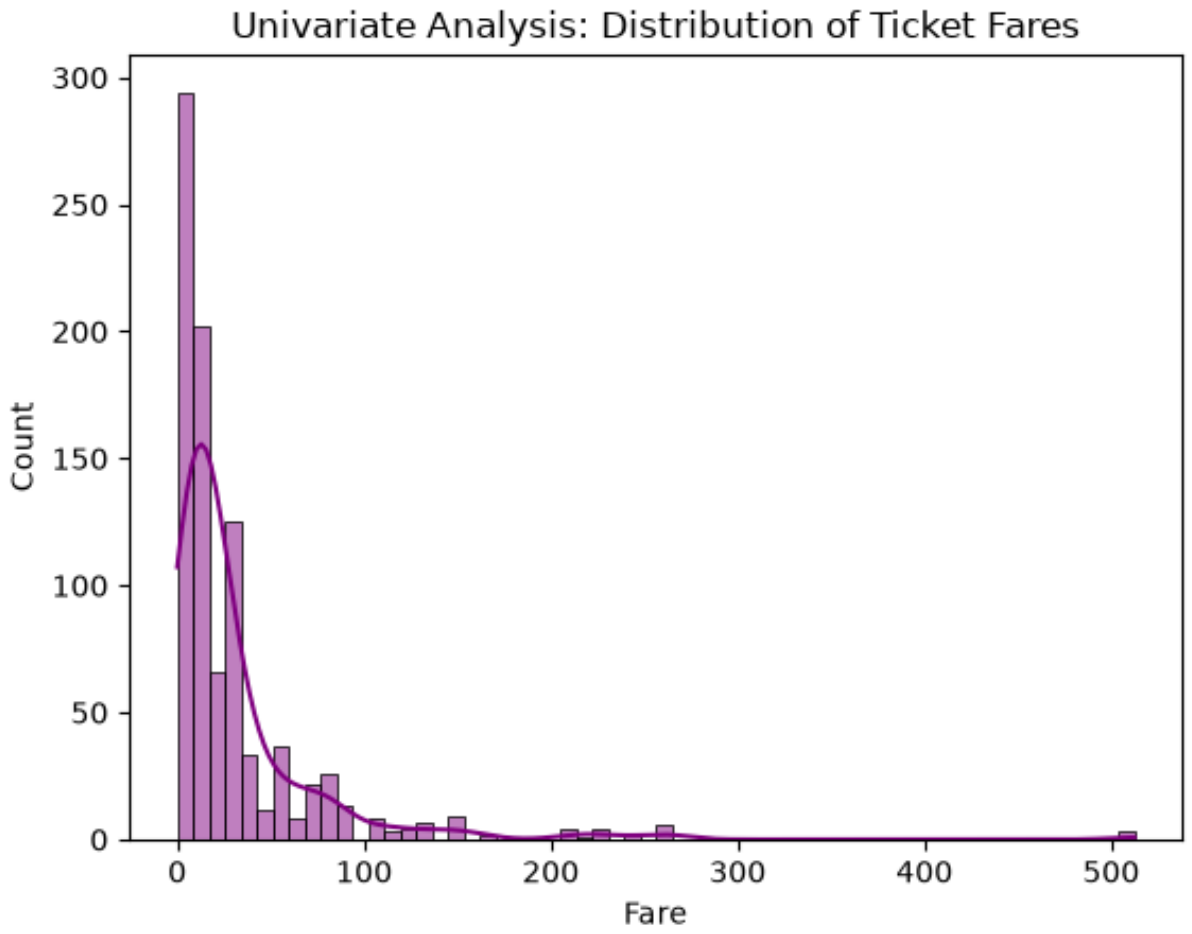

```
<Figure size 800x600 with 2 Axes>
```

```
Saved: titanic_correlation_heatmap.png
```

Code Cell 12

```
# EDA : 1. Univariate Analysis
plt.figure()
sns.histplot(df["Fare"], kde=True, color="purple")
plt.title("Univariate Analysis: Distribution of Ticket Fares")
plt.savefig("univariate_fare.png")
plt.show()
plt.close()
```

Output

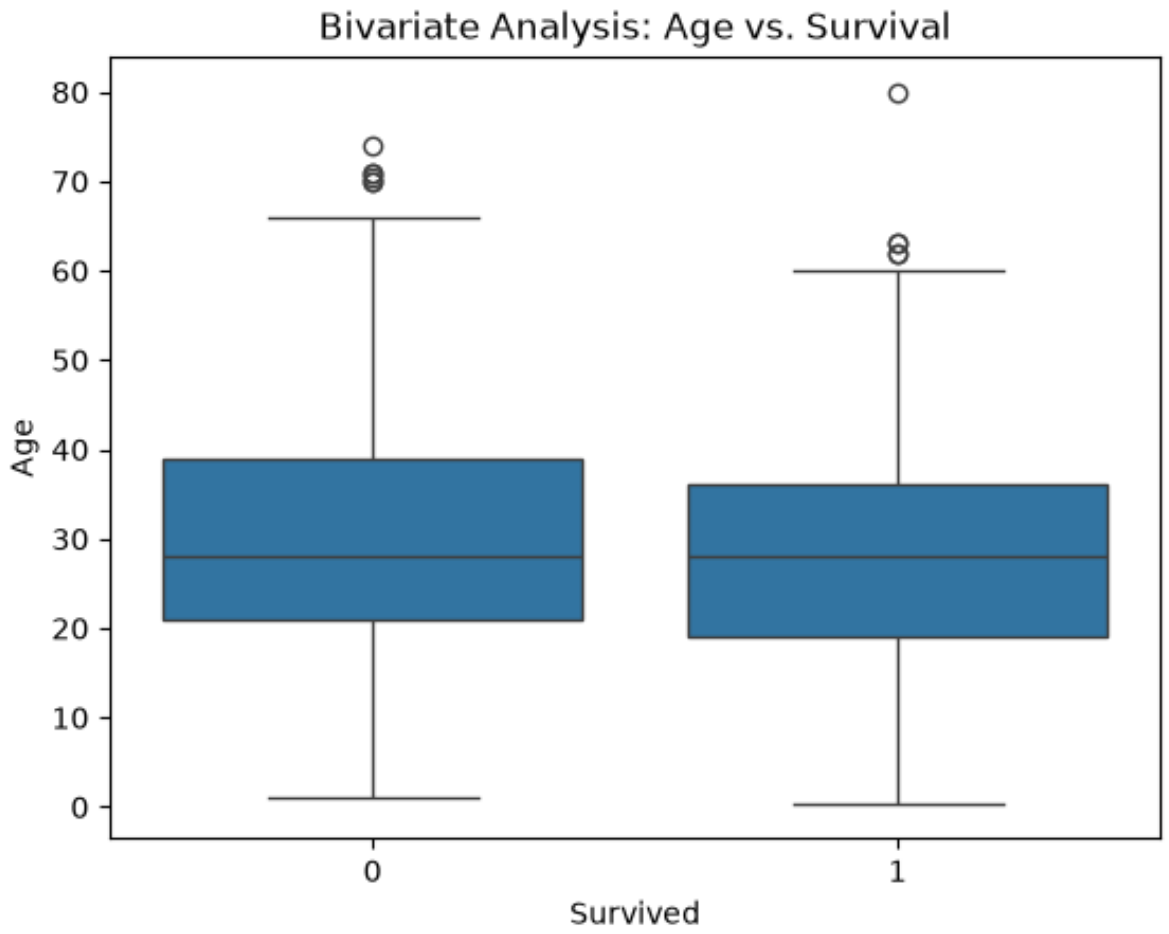



```
<Figure size 640x480 with 1 Axes>
```

Code Cell 13

```
# EDA : 2. Bivariate Analysis (Examining Relationships Between Two Variables)
sns.boxplot(data=df, x="Survived", y="Age")
plt.title("Bivariate Analysis: Age vs. Survival")
plt.savefig("bivariate_age_survival.png")
plt.show()
plt.close()
```

Output

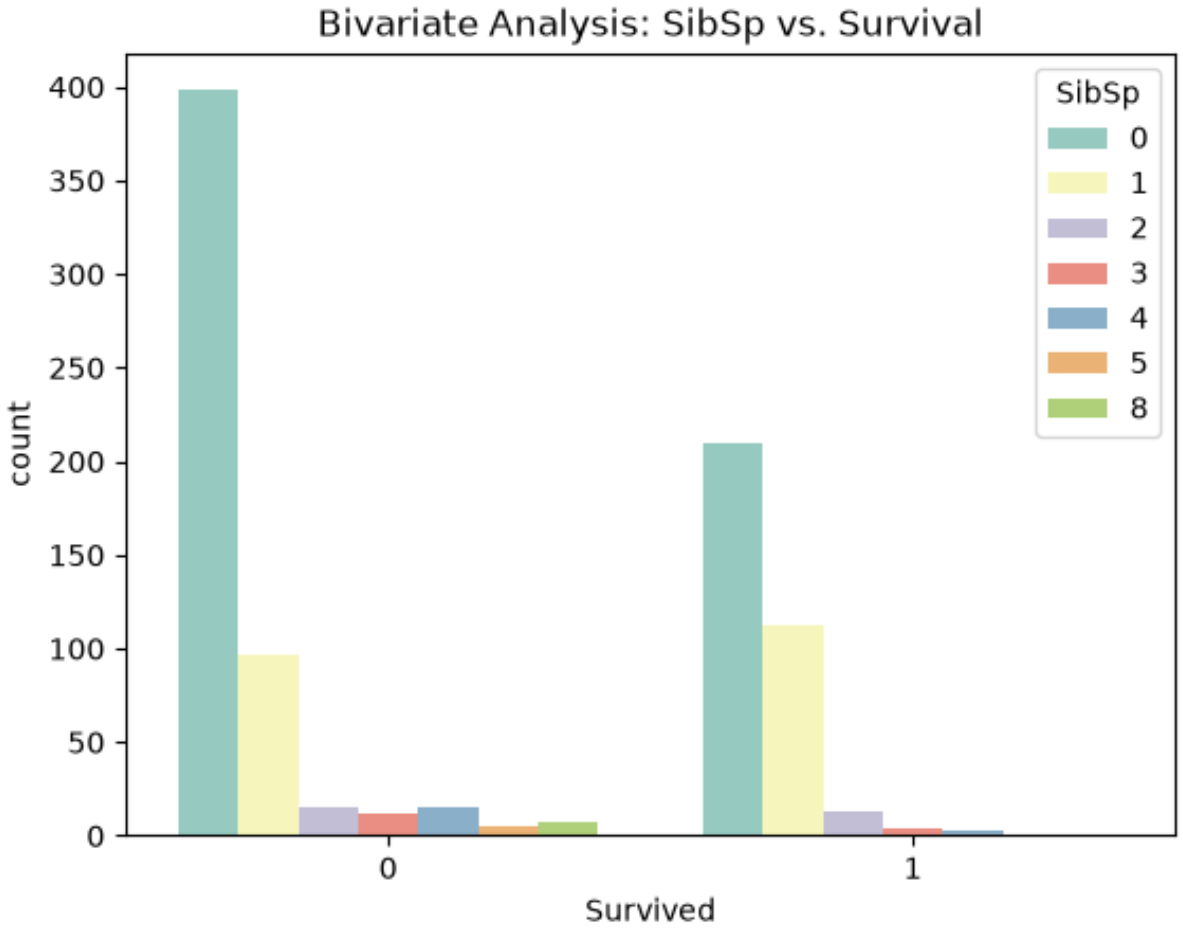



```
<Figure size 640x480 with 1 Axes>
```

Code Cell 14

```
sns.countplot(data=df, x="Survived", hue="SibSp", palette="Set3")
plt.title("Bivariate Analysis: SibSp vs. Survival")
plt.savefig("bivariate_age_survival_SibSp.png")
plt.show()
plt.close()
```

Output

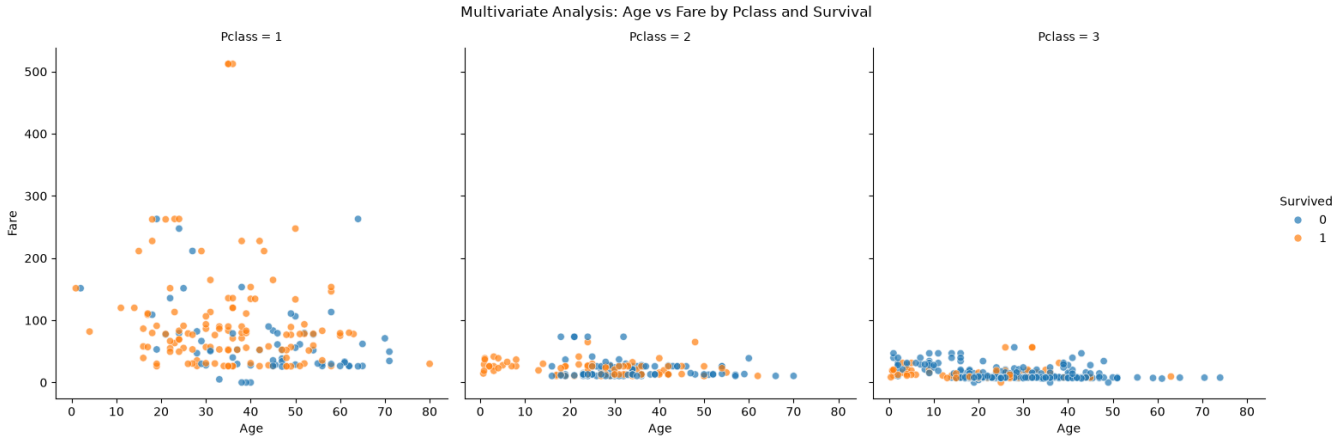



```
<Figure size 640x480 with 1 Axes>
```

Code Cell 15

```
# EDA : 3. Multivariate Analysis
g = sns.relplot(data=df,    x="Age", y="Fare", hue="Survived", col="Pclass",kind="scatter",alpha=0.7)
g.fig.suptitle("Multivariate Analysis: Age vs Fare by Pclass and Survival", y=1.03)
plt.savefig("multivariate_age_fare_pclass.png")
plt.show()
plt.close()
```

Output



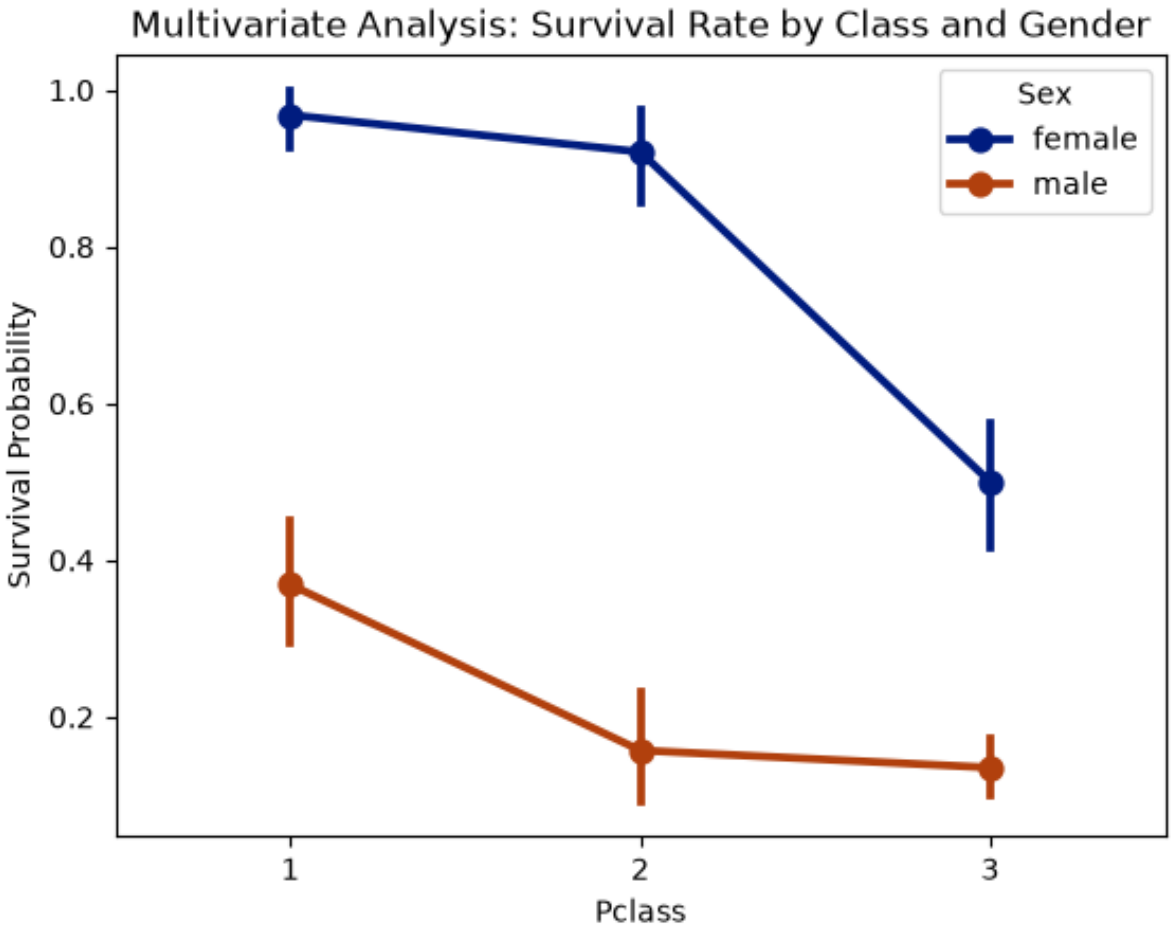

```
<Figure size 1573.11x500 with 3 Axes>
```

Code Cell 16

```
plt.figure()
```

```
sns.pointplot(data=df, x="Pclass", y="Survived", hue="Sex", palette="dark")
plt.title("Multivariate Analysis: Survival Rate by Class and Gender")
plt.ylabel("Survival Probability")
plt.savefig("multivariate_class_sex_survival.png")
plt.show()
plt.close()
```

Output




```
<Figure size 640x480 with 1 Axes>
```

Code Cell 17

```
print("FEATURE ENGINEERING & PREPROCESSING")

# Feature Engineering: Create Family Size and IsAlone features
df["family_size"] = df["SibSp"] + df["Parch"] + 1
df["is_alone"] = (df["family_size"] == 1).astype(int)

print("Engineered new features: 'family_size' and 'is_alone'")
```

Output

```
FEATURE ENGINEERING & PREPROCESSING
Engineered new features: 'family_size' and 'is_alone'
```

Code Cell 18

```
# Drop columns with excessive missing data or redundant representations
# ('deck' has too many missing values; 'alive' is target duplicate; 'embark_town' mirrors 'embarked')
columns_to_drop = ["Cabin"]
df_cleaned = df.drop(columns=[col for col in columns_to_drop if col in df.columns])
```

```
(No output)
```

Code Cell 19

```
# Separate Features (X) and Target (y)
```

```
X = df_cleaned.drop(columns=["Survived"])
y = df_cleaned["Survived"]
```

```
(No output)
```

Code Cell 20

```
# Identify numerical and categorical features
numerical_cols = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = X.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

print(f"\nNumerical Columns ({len(numerical_cols)}): {numerical_cols}")
print(f"Categorical Columns ({len(categorical_cols)}): {categorical_cols}")
```

Output

```
Numerical Columns (8): ['PassengerId', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'family_size', 'is_alone']
Categorical Columns (4): ['Name', 'Sex', 'Ticket', 'Embarked']

/var/folders/gq/pn_jxryd2kv7467xp_wyc6v80000gn/T/ipykernel_25121/431992523.py:3: Pandas4Warning: For backward compatibility, 'str
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write
  categorical_cols = X.select_dtypes(
```

Code Cell 21

```
# Build preprocessing pipelines
numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ]
)
```

```
(No output)
```

Code Cell 22

```
# Combine preprocessing steps using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numerical_cols),
        ("cat", categorical_transformer, categorical_cols),
    ]
)
```

```
(No output)
```

Code Cell 23

```
# Apply preprocessing transformations
X_processed = preprocessor.fit_transform(X)

print(f"\nFinal Processed Feature Matrix Shape: {X_processed.shape}")
print("Data Preprocessing and Feature Engineering Completed Successfully!")
```

Output

```
Final Processed Feature Matrix Shape: (891, 1585)
Data Preprocessing and Feature Engineering Completed Successfully!
```

Code Cell 24

```
from pathlib import Path
from io import BytesIO
import base64
import nbformat
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
```

```
from reportlab.lib.units import inch
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Preformatted, Image

notebook_path = Path("/Users/pratikvilasgadhe/Desktop/Programming/AI_ML/practical/Practical 1.ipynb")
pdf_path = notebook_path.with_name("Practical 1 - Executed.pdf")
with notebook_path.open(encoding="utf-8") as handle:
    notebook = nbformat.read(handle, as_version=4)
styles = getSampleStyleSheet()
styles.add(ParagraphStyle(name="TitleCenter", parent=styles["Title"], alignment=TA_CENTER, spaceAfter=18))
styles.add(ParagraphStyle(name="CellLabel", parent=styles["Heading3"], textColor=colors.HexColor("#444444"), spaceBefore=10, spac
styles.add(ParagraphStyle(name="OutputLabel", parent=styles["Heading4"], textColor=colors.HexColor("#1f5f8b"), spaceBefore=5, spa
code_style = ParagraphStyle("Code", fontName="Courier", fontSize=7.2, leading=9, leftIndent=8, rightIndent=8, backColor=colors.Hex
output_style = ParagraphStyle("Output", fontName="Courier", fontSize=7.2, leading=9, leftIndent=8, rightIndent=8, backColor=colors
def add_output(story, output):
    data = output.get("data", {})
    text = output.get("text", "")
    if text:
        story.append(Preformatted(text, output_style))
    for mime, value in data.items():
        if mime in ("text/plain", "application/vnd.code.notebook.stdout", "application/vnd.code.notebook.stderr"):
            if isinstance(value, list): value = "".join(value)
            if value: story.append(Preformatted(str(value), output_style))
        elif mime == "image/png":
            image = Image(BytesIO(base64.b64decode(value)))
            image._restrictSize(7.0 * inch, 5.0 * inch)
            story.extend([image, Spacer(1, 5)])
story = [Paragraph("Practical 1: Exploratory Data Analysis and Data Visualisation", styles["TitleCenter"]), Paragraph("Executed no
cell_number = 0
for cell in notebook.cells:
    if cell.cell_type != "code": continue
    cell_number += 1
    story.append(Paragraph(f"Code Cell {cell_number}", styles["CellLabel"]))
    story.append(Preformatted("".join(cell.source), code_style))
    outputs = cell.get("outputs", [])
    if outputs:
        story.append(Paragraph("Output", styles["OutputLabel"]))
        for output in outputs: add_output(story, output)
    else: story.append(Paragraph("(No output)", output_style))
def footer(canvas, doc):
    canvas.saveState(); canvas.setFont("Helvetica", 8); canvas.setFillColor(colors.grey)
    canvas.drawString(0.6 * inch, 0.35 * inch, "Practical 1 - Executed notebook")
    canvas.drawRightString(7.9 * inch, 0.35 * inch, f"Page {doc.page}"); canvas.restoreState()
pdf = SimpleDocTemplate(str(pdf_path), pagesize=letter, rightMargin=0.55*inch, leftMargin=0.55*inch, topMargin=0.55*inch, bottomMa
pdf.build(story, onFirstPage=footer, onLaterPages=footer)
print(f"Created {pdf_path} ({pdf_path.stat().st_size} bytes, {cell_number} code cells)")
```

Output